

Synthetic Data Generation for Microservices API Testing

A Skill-Based Pipeline — Phases A through D

Deuslirio Junior

Instituto de Informática
Universidade Federal de Goiás

2026

Agenda

1. Motivation
2. Phase A — Discovery
3. Phase B — Call Graph
4. Phase C — Flow Matrix
5. Phase C.6 — Scenario Catalog
6. Phase D — Artifacts
7. Execution & Idempotency



Motivation

Why generic test data fails

API test generators (EvoMaster, Schemathesis) need **pre-existing state** to explore meaningful paths:

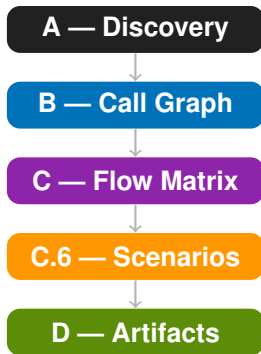
- No user in DB → every protected endpoint returns 401
- No product seeded → GET /products/{id} always 404
- Cross-service foreign keys break at runtime without the right parent record
- Generic strings (`test_user_1`) produce generic mutations

⇒ The generator explores only error paths and misses real application bugs.

⇒ A deterministic 5-phase pipeline produces auditable, editable seed artifacts from the codebase.

The pipeline at a glance

- A** Discover services, databases, identity provider
- B** Build inter-service call graph from source code
- C** Enumerate all flows (endpoint $\times$ role $\times$ state)
- C.6** Invent concrete scenarios for each success flow
- D** Generate seed SQL, auth script, Postman collection



Key property

Each phase writes a file. Re-run any phase independently. No codebase re-analysis.



Phase A — Discovery

Phase A — read compose, extract maps

Input: `docker-compose.yml` + IdP config

Steps:

1. Read each `services:` entry
2. Classify: DB, IdP, or App service
3. Extract DB name from `SPRING_DATASOURCE_URL` path segment
4. Read IdP type, realm, and roles from config or realm export

Output — three maps:

- **App** → **DB:** `order-service` → `order`, `cart-service` → `cart`
- **Identity provider:** type, realm, admin credentials, url
- **Realm roles:** CUSTOMER, ADMIN, GUEST ...



Phase B — Call Graph

Phase B — trace outbound HTTP calls in source

Input: source files of every app service

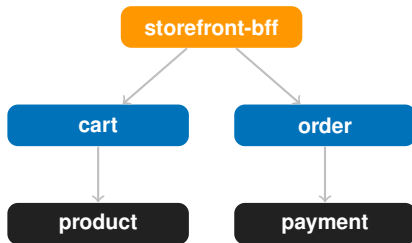
Grep patterns:

Listing 1: Spring (Java)

```
1 grep -r "@FeignClient\\|  
    RestTemplate\\  
2    \\|WebClient" services/
```

Listing 2: Node/TS

```
IN grep -r "axios\\.\\.|fetch("  
UR services/
```



Orchestrator: ≥ 2 outbound — test focus

Core: ≥ 2 inbound, few outbound

Terminal: no outbound to app services



Phase C — Flow Matrix

Phase C — enumerate endpoints × roles × states

For each service in the graph:

1. List all controller endpoints
2. Detect required role per endpoint
3. Cross-product: endpoint × role × data state

Listing 3: Find role annotations

```
1 grep -r "@PreAuthorize\  
2  \|\@Secured\  
3  \|\HttpSecurity" services/
```

Four data states per endpoint:

- success — correct role + data exists
- unauthorized — wrong role token
- unauthenticated — no token
- not_found — missing entity

Output: flow-matrix.md

Flow name	Role	State
cart_add_success	CUST	success
cart_add_unauth	ADMIN	unauth
cart_add_notoken	—	unauthn
cart_item_notfound	CUST	404



Phase C.6 — Scenario Catalog

Phase C.6 — narratives + seed tables

For each `success` row, write:

1. A 2–4 sentence **narrative** (real name, real values)
2. A **seed requirements table** (every entity that must exist)

“Alice Chen adds 2 Retro Denim Jackets (SKU: JKT-DEN-M, \$89.90) to her cart and applies coupon FALL20 for 20% off.”

Skip failure rows — absence of state IS the test condition.

Seed table format:

DB	Entity	Key values
product	brand	id=1, name='UrbanCo'
product	product	id=1, sku='JKT-DEN-M'
location	address	id=1, zip='97201'
cart	cart	id=1, cust=ALICE_UUID
promo	coupon	code='FALL20', pct=20

- Sequential integer IDs — no collisions across all scenarios
- IdP UUIDs → placeholder ALICE_UUID (patched by Phase D1)
- Output: `scenarios.md`



Phase D — Artifacts

Phase D — four artifacts from one source

Input: `scenarios.md` + `flow-matrix.md`

Sub	Artifact	Purpose
D1	<code>setup-auth.sh</code>	Create IdP users, capture UUIDs, patch seed placeholders
D2	<code>seed-all.sql</code>	All business data in one DB call using schema switches
D3	<code>postman_seed.json</code>	Postman v2.1 corpus fed to the test generator
D4	<code>docker-compose.test.yml</code>	Isolated test database with port offset

FK ordering rule

D1 — IdP users + UUID auto-patch

Problem: user UUIDs are assigned at runtime — cannot be hardcoded in seed SQL.

Pattern:

1. Create user via IdP REST API
2. Capture UUID from response
3. `sed -i` replaces `ALICE_UUID` in all seed files immediately

After `setup-auth.sh` runs, seed files are ready to apply with no editing.

Listing 4: D1 pattern (Keycloak)

```
1 create_user() {
2     local USERNAME="$1"
3     local PLACEHOLDER="$2"
4
5     USER_ID=$(curl -s ... \
6         | python3 -c "import sys,
7             json;\
8             print(json.load(sys.\
9                 stdin)\
10                [0]['id'])")
11
12     for f in "${SEED_FILES[@]};
13     do
14         sed -i \
```

D2 — single file, multiple databases

One `psql` call switches databases with `\c`. One `mysql` call uses `USE`. Seed sections follow topological order: no-FK entities first.

Listing 5: Apply in one command

```
1 # PostgreSQL
2 PGPASSWORD=p psql -h localhost -U admin -d postgres -f seed
   -all.sql
3
4 # MySQL
5 mysql -h127.0.0.1 -P3306 -uroot -p'pass' < seed-all.sql
```

Note: DB named order? Quote it: `\c "order"` or `USE 'order'`

D3 & D4 — Postman corpus + test compose

D3 — `postman_seed.json`

- One folder per service / role
- Login request first — JWT captured into variable
- success flows: real domain values from `scenarios.md`
- unauthorized: wrong role's token
- unauthenticated: no Authorization header
- not_found: path ID never seeded

D4 — `docker-compose.test.yml`

Override fields only (extends main compose):

- Port offset: `5433:5432` avoids conflicts with local DB
- Stricter healthcheck for test reliability
- Optionally mounts `seed-all.sql` as init script

Run: `java -jar evomaster.jar`
`--blackBox true --seedTestCases`
`true --seedTestCasesPath`
`postman_seed.json`



Execution & Idempotency

Resume logic — skip what already exists

Each phase checks for its output file before running:

State	Behavior
No output files	Run all A → D
flow-matrix.md exists	Skip A–C, resume C.6
scenarios.md exists	Skip A–C.6, run D only

Practical workflow

Edit a price or name in scenarios.md, re-run Phase D — all SQL and Post-

End-to-end execution order:

Stack up (wait for schemas)

setup-auth.sh

UUIDs patched

Apply seed-all.sql

Run test generator

Thank you

Questions welcome

deuslirio.junior@gmail.com

INF
INSTITUTO DE
INFORMÁTICA



UFG
UNIVERSIDADE
FEDERAL DE GOIÁS

Synthetic Data Generation for Microservices API Testing

A Skill-Based Pipeline — Phases A through D

Deuslirio Junior

Instituto de Informática
Universidade Federal de Goiás

2026